Zadanie domowe | Lekcja 11 | 2.09.2026

Laboratorium 10 — polecenia do wykonania

Przygotowanie katalogu pracy

1. Przejdź do katalogu `lab`.
2. Potwierdź bieżące położenie.
3. Wyświetl szczegółową zawartość katalogu, również pliki ukryte.
4. Utwórz katalog `work/linux-perms-lab`.
5. Skopiuj `materials/access-notes-template.md` do `work/access-notes.md`.
6. Przejdź do `work/linux-perms-lab`.
7. Potwierdź bieżące położenie.
8. Wszystkie pliki testowe twórz wyłącznie w tym katalogu.

1. Tożsamość użytkownika i grupy

1. Ustal nazwę bieżącego użytkownika.
2. Wyświetl pełną informację o jego tożsamości i członkostwie w grupach.
3. Odczytaj osobno identyfikator użytkownika.
4. Odczytaj identyfikator jego grupy głównej.
5. Odczytaj identyfikatory wszystkich grup użytkownika.
6. Wyświetl nazwy grup, do których należy użytkownik.
7. Zanotuj nazwę użytkownika, jego identyfikator, identyfikator grupy głównej i grupy dodatkowe.
8. Ustal, czy nazwa bieżącego użytkownika jest tym samym co jego identyfikator liczbowy.
9. Wskaż wynik, który pokazuje grupy dodatkowe.
10. Wyjaśnij, dlaczego podczas kontroli dostępu oceniany jest kontekst procesu, a nie osoba korzystająca z klawiatury.

Polecenie z labu	Komenda	Wynik
Nazwa użytkownika	`whoami`	`joanna`
Pełna tożsamość i grupy	`id`	`uid=1000(joanna) gid=1000(joanna)`, grupy: 1000, 4, 24, 27, 30, 46, 122, 135, 136, 997, 999
Identyfikator użytkownika	`id -u`	`1000`
Grupa główna (ID)	`id -g`	`1000`
Wszystkie grupy (ID)	`id -G`	`1000 4 24 27 30 46 122 135 136 997 999`
Nazwy grup	`id -Gn`	`joanna adm cdrom sudo dip plugdev lpadmin lxd sambashare docker vboxsf`
Zanotuj dane	`nano ~/lab/work/access-notes.md`	zapisane: `joanna`, UID 1000, GID 1000, 10 grup dodatkowych
Nazwa = identyfikator?	pytanie teoretyczne	nie: nazwa to etykieta, system używa UID
Który wynik pokazuje grupy dodatkowe?	pytanie teoretyczne	`id`, `id -G`, `id -Gn` (nie `id -g`)

Dlaczego kontekst procesu?	pytanie teoretyczne	jądro widzi UID, GID i grupy procesu, nie człowieka

2. Właściciel, grupa i uprawnienia pliku

1. Utwórz `report.txt` zawierający jeden wiersz: „dane laboratoryjne”.
2. Wyświetl szczegółowe informacje o tym pliku.
3. Odczytaj typ obiektu.
4. Odczytaj uprawnienia właściciela, grupy i pozostałych użytkowników.
5. Odczytaj właściciela oraz grupę pliku.
6. Wyświetl uprawnienia każdego elementu ścieżki prowadzącej do `report.txt`.
7. Wyjaśnij znaczenie pierwszego znaku w szczegółowym widoku pliku.
8. Wskaż część zapisu dotyczącą właściciela.
9. Ustal, czy grupa pliku musi być grupą główną jego właściciela.

Polecenie z labu	Komenda	Wynik
Utwórz `report.txt`	`echo "dane laboratoryjne" > report.txt`	plik 19 B
Szczegóły pliku	`ls -l report.txt`	`-rw-rw-r-- 1 joanna joanna 19 paź 12 23:08`
Typ obiektu	`stat -c '%F' report.txt`	plik zwykły
Uprawnienia	`stat -c '%A %a' report.txt`	`-rw-rw-r--` = `664`

Właściciel i grupa	`stat -c '%U %G' report.txt`	`joanna joanna`
Uprawnienia ścieżki	`namei -l $PWD/report.txt`	`/`, `home`, `joanna`, `lab`, `work`, `linux-perms-lab`, `report.txt`

Uprawnienia elementów ścieżki:

Element	Uprawnienia	Właściciel:grupa
`/`	`drwxr-xr-x`	root:root
`home`	`drwxr-xr-x`	root:root
`joanna`	`drwxr-x---`	joanna:joanna
`lab`	`drwxrwxr-x`	joanna:joanna
`work`	`drwxrwxr-x`	joanna:joanna
`linux-perms-lab`	`drwxrwxr-x`	joanna:joanna
`report.txt`	`-rw-rw-r--`	joanna:joanna

1. Pierwszy znak w widoku szczegółowym

- Oznacza typ obiektu: - zwykły plik, d katalog, l link symboliczny.

2. Część zapisu dotycząca właściciela

- Znaki 2-4 (np. rw-), czyli pierwsza trójka uprawnień.

3. Czy grupa pliku musi być grupą główną właściciela?

- Nie. Grupa pliku może być dowolną grupą w systemie

(np. grupą dodatkową właściciela, jeśli ma do niej prawo).

3. Znaczenie uprawnień katalogu

1. Utwórz strukturę `demo-dir/subdir`.
2. Utwórz w niej `secret.txt` zawierający wiersz „sekret testowy”.
3. Wyświetl szczegółowe informacje o obu katalogach.
4. Wyświetl szczegółowe informacje o pliku.
5. Wyświetl uprawnienia wszystkich elementów ścieżki prowadzącej do pliku.
6. Odbierz pozostałym użytkownikom prawo odczytu katalogu `demo-dir/subdir`.
7. Ponownie wyświetl uprawnienia tego katalogu.
8. Spróbuj wyświetlić listę zawartości katalogu.
9. Spróbuj odczytać `secret.txt`.
10. Przywróć pozostałym użytkownikom prawo odczytu katalogu.
11. Wyjaśnij znaczenie prawa odczytu, zapisu i przejścia dla katalogu.
12. Wyjaśnij, dlaczego prawo przejścia jest wymagane dla każdego katalogu na ścieżce.
13. Ustal, czy możliwość usunięcia pliku zależy wyłącznie od praw samego pliku.

Polecenie z labu	Komenda	Wynik
Utwórz `demo-dir/subdir`	`mkdir -p demo-dir/subdir`	oba katalogi utworzone

Utwórz `secret.txt`	`echo "sekret testowy" > demo-dir/subdir/secret.txt`	plik 15 B
Szczegóły katalogów	`ls -ld demo-dir demo-dir/subdir`	oba `drwxrwxr-x joanna joanna`
Szczegóły pliku	`ls -l demo-dir/subdir/secret.txt`	`-rw-rw-r-- joanna joanna`
Uprawnienia ścieżki	`namei -l $PWD/demo-dir/subdir/secret.txt`	`/`, `home`, `joanna`, `lab`, `work`, `linux-perms-lab`, `demo-dir`, `subdir`, `secret.txt`

Odbierz pozostałym odczyt `subdir`	`chmod o-r demo-dir/subdir`	
Ponownie wyświetl uprawnienia	`ls -ld demo-dir/subdir`	`drwxrwx--x`
Spróbuj wyświetlić listę	`ls demo-dir/subdir`	działa: `secret.txt` (jesteś właścicielką, masz `rwx`)
Spróbuj odczytać plik	`cat demo-dir/subdir/secret.txt`	działa: `sekret testowy`

Przywróć odczyt pozostałym	`chmod o+r demo-dir/subdir`	
Sprawdź uprawnienia	`ls -ld demo-dir/subdir`	`drwxrwxr-x`

1. Znaczenie praw dla katalogu

- r (odczyt): można wyświetlić listę nazw w katalogu (ls).
- w (zapis): można tworzyć, usuwać i zmieniać nazwy wpisów w katalogu (wymaga też x).
- x (przejście): można wejść do katalogu i dostać się do wpisu o znanej nazwie (cd, otwarcie pliku po ścieżce).

2. Dlaczego x jest wymagane dla każdego katalogu na ścieżce

- System sprawdza prawa po kolei, od / do pliku.
- Brak x na jednym katalogu blokuje dostęp do wszystkiego pod nim, nawet jeśli sam plik ma pełne prawa.

3. Czy możliwość usunięcia pliku zależy tylko od praw pliku?

- Nie. Usunięcie to zmiana wpisu w katalogu, więc zależy od praw katalogu (w + x).
- Prawa samego pliku nie decydują (wyjątek: sticky bit na katalogu, sekcja 8).

4. Numeryczna i symboliczna zmiana uprawnień

1. Utwórz pliki `private.txt`, `shared.txt`, `public.txt` i `script.sh`.
2. Ustaw `private.txt` tak, aby właściciel mógł go czytać i modyfikować, a pozostali nie mieli dostępu.
3. Ustaw `shared.txt` tak, aby właściciel mógł go czytać i modyfikować, grupa czytać, a pozostali nie mieli dostępu.
4. Ustaw `public.txt` tak, aby właściciel mógł go czytać i modyfikować, a grupa i pozostali tylko czytać.
5. Ustaw `script.sh` tak, aby pełny dostęp miał wyłącznie właściciel.
6. Wykonaj powyższe zmiany za pomocą zapisu numerycznego.
7. Wyświetl i sprawdź rezultaty.
8. Za pomocą zapisu symbolicznego dodaj właścicielowi `script.sh` prawo wykonania.
9. Odbierz pozostałym użytkownikom prawo odczytu `public.txt`.
10. Dodaj grupie prawo zapisu do `shared.txt`.
11. Ustaw symbolicznie `private.txt` tak, aby właściciel miał odczyt i zapis, grupa tylko odczyt, a pozostali nie mieli dostępu.
12. Ponownie wyświetl i sprawdź rezultaty.
13. Wyjaśnij, kiedy wygodniejszy jest zapis numeryczny, a kiedy symboliczny.
14. Wyjaśnij ryzyko rekurencyjnej zmiany praw.
15. Wskaż bezpieczną kolejność diagnozowania problemu dostępu: kontekst użytkownika, prawa pliku i prawa wszystkich elementów ścieżki.

Polecenie z labu	Komenda	Wynik
Utwórz 4 pliki	`touch private.txt shared.txt public.txt script.sh`	4 puste pliki
`private.txt`: właściciel rw, reszta nic	`chmod 600 private.txt`	`-rw-------`
`shared.txt`: właściciel rw, grupa r	`chmod 640 shared.txt`	`-rw-r-----`

`public.txt`: właściciel rw, reszta r	`chmod 644 public.txt`	`-rw-r--r--`
`script.sh`: tylko właściciel pełny	`chmod 700 script.sh`	`-rwx------`
Dodaj właścicielowi `x` w `script.sh`	`chmod u+x script.sh`	bez zmian (`-rwx------`)
Odbierz pozostałym odczyt `public.txt`	`chmod o-r public.txt`	`-rw-r-----`
Dodaj grupie zapis `shared.txt`	`chmod g+w shared.txt`	`-rw-rw----`
`private.txt` symbolicznie: właściciel rw, grupa r	`chmod u=rw,g=r,o= private.txt`	`-rw-r-----`

1. Zapis numeryczny a symboliczny

- Numeryczny (np. 640): wygodny, gdy ustawiam komplet uprawnień od zera jednym poleceniem.
- Symboliczny (np. g+w, o-r): wygodny, gdy zmieniam jedno prawo i nie chcę ruszać pozostałych.

2. Ryzyko rekurencyjnej zmiany praw (chmod -R)

- Jedno polecenie zmienia prawa całego drzewa naraz, więc błąd w ścieżce lub w trybie trudno cofnąć.

- Pliki i katalogi potrzebują różnych praw (katalogi x, pliki zwykle nie).

  chmod -R 644 odbiera x katalogom i blokuje wejście.

- Zmiana na złym katalogu (np. /, /etc, /home) może zepsuć system

  lub otworzyć dostęp do prywatnych danych.

3. Bezpieczna kolejność diagnozowania problemu z dostępem

- 1) Kontekst użytkownika: whoami, id (UID, GID, grupy procesu).

- 2) Prawa pliku: ls -l, stat (właściciel, grupa, tryb).

- 3) Prawa wszystkich elementów ścieżki: namei -l (brak x na którymkolwiek

  katalogu blokuje dostęp).

- Dopiero potem poprawka, możliwie najmniejsza.

5. Diagnostyka odmowy dostępu

1. Utwórz strukturę `denied/a/b`.
2. Utwórz w niej `file.txt` zawierający wiersz „test”.
3. Ustaw katalog `denied/a` tak, aby pełny dostęp miał wyłącznie właściciel.
4. Wyświetl szczegółowe informacje o katalogach `denied`, `denied/a` i `denied/a/b`.
5. Wyświetl szczegółowe informacje o pliku.
6. Wyświetl uprawnienia wszystkich elementów ścieżki prowadzącej do pliku.
7. Wskaż element ścieżki, który mógłby spowodować odmowę dostępu dla innego użytkownika.
8. Przywróć dla `denied/a` pełny dostęp właściciela oraz odczyt i przejście dla grupy i pozostałych.
9. Odpowiedz, czy problem dostępu zawsze wynika z praw końcowego pliku.
10. Wyjaśnij, jak analiza całej ścieżki pomaga znaleźć przyczynę.

Polecenie z labu	Komenda	Wynik
Utwórz `denied/a/b`	`mkdir -p denied/a/b`	3 poziomy katalogów

Utwórz `file.txt`	`echo "test" > denied/a/b/file.txt`	plik z wierszem `test`
`denied/a`: tylko właściciel	`chmod 700 denied/a`	`drwx------`
Uprawnienia ścieżki	`namei -l $PWD/denied/a/b/file.txt`	`a` = `drwx------`, reszta `drwxrwxr-x`, plik `-rw-rw-r--`
Element, który zablokuje innego użytkownika	analiza	`denied/a` (brak `x` dla pozostałych)
Przywróć `denied/a`	`chmod 755 denied/a`	
Sprawdź uprawnienia	`ls -ld denied/a`	`drwxr-xr-x`

1. Element ścieżki powodujący odmowę dla innego użytkownika

- denied/a (drwx------): pozostali nie mają x, więc nie przejdą do b

  ani do file.txt.

2. Czy problem dostępu zawsze wynika z praw końcowego pliku?

- Nie. Plik file.txt jest czytelny dla wszystkich (rw-rw-r--),

a mimo to dostęp jest zablokowany przez katalog a.

3. Jak analiza całej ścieżki pomaga znaleźć przyczynę

- namei -l pokazuje prawa każdego elementu od / do pliku.

- Dzięki temu widać pierwszy katalog bez x i wiadomo, co poprawić,

zamiast zmieniać prawa pliku.

6. Grupy projektowe i zmiana właściciela grupowego

1. Utwórz katalog `projekt`.
2. Utwórz w nim `notes.txt` zawierający wiersz „notatka projektu”.
3. Wyświetl szczegółowe informacje o katalogu i pliku.
4. Jeżeli prowadzący wyrazi zgodę, utwórz grupę `dev-lab`.
5. Dodaj bieżącego użytkownika do tej grupy bez usuwania go z dotychczasowych grup.
6. Sprawdź grupy widoczne przez bieżący proces.
7. Sprawdź członkostwo zapisane dla konta bieżącego użytkownika.
8. Jeśli wyniki są różne, rozpocznij nową sesję i sprawdź je ponownie.
9. Zmień grupę katalogu `projekt` na `dev-lab`.
10. Ustaw katalog tak, aby właściciel i grupa mieli pełny dostęp, pozostali nie mieli dostępu, a nowe elementy dziedziczyły grupę katalogu.
11. Wyświetl i sprawdź uprawnienia katalogu.
12. Jeżeli nie tworzysz grupy, przeanalizuj ćwiczenie na podstawie `materials/permission-scenarios.md`.
13. Wyjaśnij różnicę między zmianą właściciela a zmianą grupy.
14. Wskaż specjalny bit użyty w ustawieniach katalogu.
15. Wyjaśnij, dlaczego zmiana członkostwa w grupie może wymagać nowej sesji.

Polecenie z labu	Komenda	Wynik
Utwórz `projekt`	`mkdir projekt`	`drwxrwxr-x joanna joanna`

Utwórz `notes.txt`	`echo "notatka projektu" > projekt/notes.txt`	17 B, `-rw-rw-r--`
Szczegóły katalogu i pliku	`ls -ld projekt`, `ls -l projekt/notes.txt`	patrz wyżej
Utworzenie grupy `dev-lab`, dodanie do niej, zmiana grupy katalogu	pominięte	pominięte
Analiza zamiast grupy	`less materials/permission-scenarios.md`	scenariusz 2 omówiony

1. Zmiana właściciela a zmiana grupy

- chown zmienia właściciela (chown user plik), chgrp zmienia grupę (chgrp grupa plik). Prawa właściciela i grupy to osobne trójki.

2. Specjalny bit w ustawieniach katalogu

- SGID (g+s, tryb 2770): nowe elementy dziedziczą grupę katalogu.

3. Dlaczego zmiana członkostwa w grupie może wymagać nowej sesji

- Grupy procesu są ustalane przy logowaniu i dziedziczone przez procesy potomne. Zmiana w /etc/group nie zmienia procesów już uruchomionych, dopiero nowa sesja czyta grupy ponownie.

7. Dziedziczenie grupy w katalogu projektowym

1. Sprawdź szczegółowe uprawnienia katalogu `projekt`.
2. Ustal, czy w części grupowej widoczny jest specjalny bit dziedziczenia grupy.
3. Utwórz w katalogu `projekt` plik `new-file.txt` zawierający wiersz „plik w projekcie”.
4. Wyświetl właściciela, grupę i uprawnienia nowego pliku.
5. Porównaj grupę nowego pliku z grupą katalogu.
6. Ustal, czy dziedziczenie grupy automatycznie nadało grupie prawo zapisu.
7. Wyjaśnij, jak maska domyślnych uprawnień mogła wpłynąć na rezultat.

Polecenie z labu	Komenda	Wynik
Szczegółowe uprawnienia `projekt`	`ls -ld projekt`	`drwxrws--- joanna joanna`
Czy widoczny bit dziedziczenia grupy?	analiza	tak, `s` w części grupowej (SGID)
Utwórz `new-file.txt`	`echo "plik w projekcie" > projekt/new-file.txt`	plik 17 B
Właściciel, grupa, uprawnienia pliku	`ls -l`, `stat -c '%U %G %A'`	`joanna joanna -rw-rw-r--`
Porównaj grupę pliku z grupą katalogu	`stat -c '%G' projekt`	obie `joanna`
Czy dziedziczenie nadało grupie zapis?	analiza	nie, SGID dziedziczy tylko grupę

Wpływ maski `umask` `0002`

1. Czy w części grupowej widać bit dziedziczenia grupy?

- Tak. projekt ma drwxrws---, litera s w części grupowej to SGID.

- SGID ustawiono ręcznie (chmod 2770 projekt), bo tworzenie grupy dev-lab pominięto.

2. Grupa nowego pliku a grupa katalogu

- Obie to joanna (SGID działa, ale grupa katalogu to nadal joanna, więc efekt jest mało widoczny).

3. Czy dziedziczenie grupy nadało grupie prawo zapisu?

- Nie. SGID dziedziczy tylko grupę, nie uprawnienia.

- Zapis grupy (rw-) wynika z umask 0002: 666 minus 002 = 664.

4. Jak maska wpłynęła na wynik

- Przy umask 0002 nowe pliki mają 664, grupa ma zapis.

- Przy umask 0022 byłoby 644 i grupa nie miałaby zapisu, mimo SGID.

8. Sticky bit w katalogu współdzielonym

1. Wyświetl szczegółowe uprawnienia katalogu `/tmp`.
2. Znajdź specjalny znak na końcu zapisu uprawnień.
3. Utwórz w katalogu laboratoryjnym katalog `shared`.
4. Ustaw go jako katalog dostępny do odczytu, zapisu i przejścia dla wszystkich, ale ogranicz możliwość usuwania cudzych plików.
5. Wyświetl i sprawdź jego uprawnienia.

6. Wyjaśnij znaczenie specjalnego znaku widocznego na końcu uprawnień katalogu.
7. Wyjaśnij, dlaczego takie ustawienie jest ważne w `/tmp`.
8. Ustal, czy chroni ono przed wszystkimi zagrożeniami związanymi z plikami tymczasowymi.

Polecenie z labu	Komenda	Wynik
Szczegółowe uprawnienia `/tmp`	`ls -ld /tmp`	`drwxrwxrwt root root`
Znajdź specjalny znak na końcu	analiza	`t` (sticky bit)
Utwórz katalog `shared`	`mkdir shared`	katalog utworzony
`rwx` dla wszystkich, ogranicz usuwanie cudzych plików	`chmod 1777 shared`	ustawione
Wyświetl uprawnienia	`ls -ld shared`	`drwxrwxrwt joanna joanna`

1. Znaczenie znaku t na końcu uprawnień katalogu

- To sticky bit. W katalogu, w którym wszyscy mają zapis, plik może usunąć lub przemianować tylko jego właściciel, właściciel katalogu albo root.

2. Dlaczego to ważne w /tmp

- /tmp jest zapisywalny dla wszystkich użytkowników.

- Bez sticky bitu każdy mógłby usunąć lub podmienić cudze pliki tymczasowe (np. pliki innych programów i użytkowników).

3. Czy sticky bit chroni przed wszystkimi zagrożeniami plików tymczasowych?

- Nie. Chroni tylko przed usuwaniem i przemianowaniem cudzych plików.
- Nie chroni przed odczytem, jeśli plik ma szerokie prawa odczytu, ani przed przewidywalnymi nazwami plików (ataki typu symlink i wyścig). Pliki tymczasowe powinny mieć losowe nazwy i ścisłe prawa (np. mktemp, tryb 600).

9. Rozpoznawanie specjalnych bitów plików

1. Wyświetl szczegółowe informacje o `/usr/bin/passwd`.
2. Sprawdź, czy w uprawnieniach właściciela widoczny jest specjalny bit.
3. Wyszukaj w `/usr/bin` wszystkie zwykłe pliki z takim bitem.
4. Wyświetl szczegółowe informacje o znalezionych plikach.
5. Ukryj komunikaty wynikające z braku uprawnień podczas wyszukiwania.
6. Nie skanuj całego systemu bez zalecenia prowadzącego.
7. Wyjaśnij działanie specjalnego bitu dla pliku wykonywalnego.
8. Wyjaśnij, dlaczego program uruchamiany z efektywnymi uprawnieniami administratora jest szczególnie wrażliwy.
9. Ustal, czy sama obecność specjalnego bitu oznacza podatność.

Polecenie z labu	Komenda	Wynik
Szczegóły `/usr/bin/passwd`	`ls -l /usr/bin/passwd`	`-rwsr-xr-x 1 root root 59976`
Czy u właściciela widać specjalny bit?	analiza	tak, `s` na miejscu `x` (SUID)

Znajdź zwykłe pliki z SUID w `/usr/bin`	`find /usr/bin -type f -perm -4000 2>/dev/null`	11 plików (lista wyżej)
Szczegóły znalezionych plików	`find /usr/bin -type f -perm -4000 -exec ls -l {} + 2>/dev/null`	wszystkie `-rwsr-xr-x root root`
Ukryj komunikaty o braku uprawnień	`2>/dev/null`	użyte w obu komendach
Nie skanuj całego systemu	ograniczenie do `/usr/bin`	zachowane

Znalezione pliki z SUID w /usr/bin (11): chfn, chsh, fusermount3,

gpasswd, mount, newgrp, passwd, pkexec, su, sudo, umount.

Wszystkie: -rwsr-xr-x root root.

1. Działanie SUID dla pliku wykonywalnego

- Program uruchamia się z uprawnieniami właściciela pliku (tu root),

  a nie użytkownika, który go uruchomił.

- Dzięki temu zwykły użytkownik może wykonać wąską czynność

  wymagającą roota, np. passwd zmienia hasło w /etc/shadow.

2. Dlaczego program z efektywnymi uprawnieniami roota jest wrażliwy

- Błąd w programie (przepełnienie bufora, złe użycie zmiennych

  środowiskowych, argumentów lub ścieżek) pozwala atakującemu

wykonać kod jako root.

- Skutki błędu są więc znacznie większe niż w zwykłym programie.

3. Czy sama obecność SUID oznacza podatność?

- Nie. Wiele standardowych programów systemowych ma SUID

(passwd, sudo, su, mount).

- Problem pojawia się, gdy program jest niepotrzebny, nietypowy,

w nietypowym miejscu lub ma znaną podatność.

10. Lokalne źródła użytkowników, grup i haseł

1. Odczytaj wpis bieżącego użytkownika z systemowego źródła kont.
2. Wyświetl wszystkie grupy znane systemowi.
3. Odczytaj wpis grupy `sudo`.
4. Porównaj właścicieli i uprawnienia plików `/etc/passwd`, `/etc/group` i `/etc/shadow`.
5. Wyświetl pierwszych pięć wierszy `/etc/passwd`.
6. Spróbuj odczytać `/etc/shadow` jako zwykły użytkownik.
7. Za zgodą prowadzącego wyświetl pierwsze trzy wiersze `/etc/shadow` z uprawnieniami administracyjnymi.
8. Wyjaśnij, dlaczego `/etc/passwd` jest zwykle czytelny dla wszystkich.
9. Wyjaśnij znaczenie znaku `x` w drugim polu wpisu użytkownika.
10. Wyjaśnij, dlaczego `/etc/shadow` wymaga silniejszej ochrony.

Polecenie z labu	Komenda	Wynik
Wpis bieżącego użytkownika	`getent passwd $USER`	`joanna:x:1000:1000:...:/home/joanna:/bin/bash`
Wszystkie grupy systemu	`getent group`	lista grup (m.in. `root`, `adm`, `sudo`, `docker`, `vboxsf`)

Wpis grupy `sudo`	`getent group sudo`	`sudo:x:27:joanna`
Porównanie plików kont	`ls -l /etc/passwd /etc/group /etc/shadow`	`passwd`: `rw-r--r-- root root`, `group`: `rw-r--r-- root root`, `shadow`: `rw-r----- root shadow`
Pierwsze 5 wierszy `/etc/passwd`	`head -n 5 /etc/passwd`	`root`, `daemon`, `bin`, `sys`, `sync`
Odczyt `/etc/shadow` jako zwykły użytkownik	`cat /etc/shadow`	`Brak dostępu`
Pierwsze 3 wiersze `/etc/shadow` przez administratora	wymaga zgody prowadzącego	niewykonane, czeka na zgodę

1. Dlaczego /etc/passwd jest zwykle czytelny dla wszystkich

- Wiele programów zamienia UID na nazwę i odwrotnie (ls -l, id, ps),

  więc muszą go czytać zwykłe konta.

- Nie zawiera haszy haseł, więc odczyt nie ujawnia haseł.

2. Znaczenie znaku x w drugim polu wpisu użytkownika

- Hasło nie jest przechowywane w tym pliku.

- Hasz hasła jest w /etc/shadow.

3. Dlaczego /etc/shadow wymaga silniejszej ochrony

- Zawiera hasze haseł. Kto je odczyta, może je łamać offline.

- Dlatego ma rw-r----- root:shadow, zwykły użytkownik nie ma dostępu.

Uwaga: wyświetlenie pierwszych trzech wierszy /etc/shadow przez

administratora wymaga zgody prowadzącego, punkt niewykonany.

11. Dodatkowe konta z identyfikatorem administratora

1. Pobierz wszystkie wpisy ze źródła kont użytkowników.
2. Wybierz wpisy, których identyfikator użytkownika wynosi `0`.
3. Zanotuj wszystkie znalezione konta.
4. Jeśli wynik zawiera konto inne niż `root`, nie usuwaj go automatycznie i omów wynik z prowadzącym.
5. Wyjaśnij, dlaczego identyfikator `0` jest ważniejszy niż nazwa konta.
6. Ustal, czy konto o innej nazwie, ale z identyfikatorem `0`, posiada uprawnienia administratora.

Polecenie z labu	Komenda	Wynik
Pobierz wszystkie wpisy kont	`getent passwd`	pełna lista kont
Wybierz wpisy z UID 0	`getent passwd | awk -F: '$3 == 0'`	`root:x:0:0:root:/root:/bin/bash`
Zanotuj znalezione konta	analiza	jedno konto: `root`
Konto inne niż `root`	analiza	brak, nic do omawiania

Konta z UID 0: tylko root (root:x:0:0:root:/root:/bin/bash).

Konta innego niż root z UID 0 nie znaleziono.

1. Dlaczego identyfikator 0 jest ważniejszy niż nazwa konta

- Jądro sprawdza UID, a nie nazwę. Uprawnienia administratora

  wynikają z UID 0.

- Nazwa "root" to tylko etykieta. Konto o innej nazwie, ale

  z UID 0, miałoby takie same pełne prawa.

2. Czy konto o innej nazwie, ale z UID 0, ma uprawnienia administratora?

- Tak. Ma pełne uprawnienia administratora, bo liczy się UID 0.

- Dlatego dodatkowe konto z UID 0 to poważny sygnał ostrzegawczy

  (często ślad włamania lub tylnej furtki) i trzeba je omówić

  z prowadzącym, a nie usuwać automatycznie.

12. Analiza reguł uprawnień administracyjnych

1. Wyświetl reguły określające, co bieżący użytkownik może uruchamiać z uprawnieniami administracyjnymi.
2. Zanotuj, czy użytkownik może korzystać z takich uprawnień.
3. Zanotuj, czy występują reguły niewymagające hasła.
4. Zanotuj, czy występują reguły zezwalające na wszystkie operacje.
5. Wypisz dozwolone programy i zakres ich argumentów.
6. Otwórz `materials/sudoers-examples.md` w przeglądarce tekstowej.
7. Dla każdego przykładu określ, czy reguła jest wąska, czy szeroka.
8. Sprawdź, czy pozwala uruchomić edytor, interpreter albo powłokę.
9. Sprawdź, czy wymaga uwierzytelnienia.
10. Ustal, czy kontroluje argumenty programu.
11. Opisz wpływ przejęcia konta objętego daną regułą.

Odpowiedzi na pytania z pliku

- **1:** alice może tylko sprawdzić status nginx. Restartu nie pozwala (inny argument). Argumenty są ograniczone.
- **2:** szersza, bo `*` dopuszcza każdy podkomenda `systemctl` (`restart`, `stop`, `edit`...). Tak, może prowadzić do eskalacji.

- **3:** praktycznie konto administracyjne. Uzasadniona np. dla administratora systemu. Monitorowanie: logi `sudo` (`/var/log/auth.log`), `journalctl`.
- **4:** `NOPASSWD` nie zawsze jest podatnością (bywa potrzebne w automatyzacji), ale podnosi ryzyko. Sprawdzić: właściciela i prawa skryptu, czy przyjmuje argumenty, czy wywołuje inne programy po względnej ścieżce. Modyfikować skrypt może każdy, kto ma do niego zapis (lub do jego katalogu), więc powinien należeć do roota i nie być zapisywalny dla innych.
- **5:** edytor jest ryzykowny, bo ma wbudowane wywoływanie komend. Może zapisać dowolny plik (np. `/etc/passwd`) i uruchomić komendę.
- **6:** interpreter wykonuje dowolny kod, więc reguła jest równoważna bardzo szerokiemu uprawnieniu.

Polecenie z labu	Komenda	Wynik
Reguły sudo użytkownika	`sudo -l`	`(ALL : ALL) ALL`
Czy może korzystać z sudo?	analiza	tak
Reguły bez hasła?	analiza	nie (brak `NOPASSWD`)
Reguły na wszystkie operacje?	analiza	tak, `ALL`
Dozwolone programy i argumenty	analiza	wszystkie programy, dowolne argumenty
Otwórz przykłady	`less ~/lab/materials/sudoers-examples.md`	6 przykładów

Ocena każdego przykładu	analiza	tabela wyżej

Reguły joanna (sudo -l): (ALL : ALL) ALL.

- Może używać uprawnień administracyjnych: tak.
- Reguły NOPASSWD: brak (sudo pyta o hasło).
- Reguły na wszystkie operacje: tak (ALL).
- Dozwolone programy: wszystkie, dowolne argumenty.

Przykłady z sudoers-examples.md:

1. systemctl status nginx: wąska, bez edytora/interpretera, hasło wymagane, argumenty ograniczone. Skutek przejęcia: minimalny.
2. systemctl *: szeroka, argumenty nieograniczone, hasło wymagane. Skutek: kontrola nad usługami, możliwa eskalacja do roota.
3. ALL=(ALL) ALL: bardzo szeroka, hasło wymagane. Skutek: pełne przejęcie systemu.
4. NOPASSWD run-backup: wąska co do programu, bez hasła. Skutek zależy od skryptu: sprawdzić właściciela, prawa i argumenty, skrypt nie może być zapisywalny dla innych.
5. vim jako root: pozornie wąska, w praktyce szeroka. Edytor zapisze dowolny plik i uruchomi komendę (powłokę). Skutek: pełny root.
6. python3 jako root: interpreter wykonuje dowolny kod, równoważne bardzo szerokiemu uprawnieniu. Skutek: pełny root.

13. Mini-scenariusze diagnostyczne

1. Otwórz `materials/permission-scenarios.md` w przeglądarce tekstowej.
2. Dla każdego scenariusza zapisz w `work/access-notes.md` ustalone fakty.
3. Zapisz prawdopodobną przyczynę problemu.
4. Zaproponuj polecenia diagnostyczne bez wykonywania niepotrzebnych zmian.
5. Zaproponuj najmniejszą bezpieczną poprawkę.
6. Zapisz, czego nie należy robić w danym przypadku.
7. W diagnozie uwzględnij tożsamość i grupy procesu.
8. Uwzględnij właściciela, grupę i uprawnienia pliku.
9. Uwzględnij uprawnienia katalogu zawierającego plik.
10. Uwzględnij wszystkie elementy ścieżki prowadzącej do pliku.
11. Jeśli scenariusz dotyczy podwyższonych uprawnień, uwzględnij reguły użytkownika.

Polecenie z labu	Komenda	Wynik
Otwórz scenariusze	`less ~/lab/materials/permission-scenarios.md`	6 scenariuszy
Zapisz fakty dla każdego scenariusza	`nano ~/lab/work/access-notes.md`	6 wpisów w pliku (`grep -c "=== Scenariusz"` = 6)
Prawdopodobna przyczyna	analiza	pole `Hipoteza` w każdym wpisie
Polecenia diagnostyczne bez zmian	analiza	pole `Diagnostyka` (m.in. `id`, `namei -l`, `ls -ld`, `getent group`)
Najmniejsza bezpieczna poprawka	analiza	pole `Poprawka` i `Dlaczego minimalna`
Czego nie robić	analiza	pole `Czego nie robić`
Tożsamość i grupy procesu	analiza	pole `Kontekst`
Właściciel, grupa, prawa pliku i katalogu, cała ścieżka	analiza	pola `Obiekt` i `Ścieżka`
Reguły sudo (dla scenariusza z podwyższonymi uprawnieniami)	analiza	pole `Diagnoza`, scenariusz 6